

```
In [3]: import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, roc_auc_score

from sklearn.ensemble import RandomForestClassifier
from sklearn.neural_network import MLPClassifier
from xgboost import XGBClassifier
```

```
In [4]: df = pd.read_csv("creditcard.csv")
df.head()
```

Out[4]:

	Time	V1	V2	V3	V4	V5	
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	(
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	'
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	'
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	(

5 rows × 31 columns



```
In [5]: df['Class'].value_counts()
```

Out[5]: Class
0 284315
1 492
Name: count, dtype: int64

```
In [6]: X = df.drop("Class", axis=1)
y = df["Class"]
```

```
In [7]: scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
In [8]: X_train, X_test, y_train, y_test = train_test_split(
    X_scaled,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

```
In [9]: rf = RandomForestClassifier(
        n_estimators=100,
        random_state=42,
        class_weight="balanced"
    )

    rf.fit(X_train, y_train)
    y_pred_rf = rf.predict(X_test)
```

```
In [10]: print("Random Forest Results")
        print(classification_report(y_test, y_pred_rf))

        rf_auc = roc_auc_score(y_test, rf.predict_proba(X_test)[: , 1])
        print("ROC-AUC:", rf_auc)
```

```
Random Forest Results
              precision    recall  f1-score   support

     0           1.00        1.00        1.00     56864
     1           0.96        0.74        0.84         98

 accuracy              1.00        56962
 macro avg           0.98        0.87        0.92        56962
 weighted avg        1.00        1.00        1.00        56962

ROC-AUC: 0.9529092148254912
```

```
In [11]: xgb = XGBClassifier(
        scale_pos_weight=(y_train == 0).sum() / (y_train == 1).sum(),
        eval_metric='logloss',
        random_state=42
    )

    xgb.fit(X_train, y_train)
    y_pred_xgb = xgb.predict(X_test)
```

```
In [12]: print("XGBoost Results")
        print(classification_report(y_test, y_pred_xgb))

        xgb_auc = roc_auc_score(y_test, xgb.predict_proba(X_test)[: , 1])
        print("ROC-AUC:", xgb_auc)
```

```
XGBoost Results
              precision    recall  f1-score   support

     0           1.00        1.00        1.00     56864
     1           0.88        0.84        0.86         98

 accuracy              1.00        56962
 macro avg           0.94        0.92        0.93        56962
 weighted avg        1.00        1.00        1.00        56962

ROC-AUC: 0.9682378578893572
```

```

In [14]: mlp = MLPClassifier(
            hidden_layer_sizes=(32, 16),
            activation='relu',
            solver='adam',
            max_iter=50,
            random_state=42
        )

        mlp.fit(X_train, y_train)
        y_pred_mlp = mlp.predict(X_test)

In [15]: print("Neural Network (MLP) Results")
        print(classification_report(y_test, y_pred_mlp))

        mlp_auc = roc_auc_score(y_test, mlp.predict_proba(X_test)[: , 1])
        print("ROC-AUC:", mlp_auc)

```

```

Neural Network (MLP) Results
              precision    recall  f1-score   support

     0           1.00        1.00        1.00     56864
     1           0.82        0.83        0.82         98

 accuracy              1.00        56962
 macro avg           0.91        0.91        0.91        56962
 weighted avg        1.00        1.00        1.00        56962

ROC-AUC: 0.9754798057377143

```

```

In [16]: results_df = pd.DataFrame({
            "Model": ["Random Forest", "XGBoost", "Yapay Sinir Ağı (MLP)"],
            "ROC-AUC": [rf_auc, xgb_auc, mlp_auc]
        })

        results_df

```

Out[16]:

	Model	ROC-AUC
0	Random Forest	0.952909
1	XGBoost	0.968238
2	Yapay Sinir Ağı (MLP)	0.975480